University Name

Bachelor Thesis

Benchmarking Large Language Models on Algorithmic Programming Tasks

A Case Study Using Advent of Code with Python and Rust

Your Name

Supervisor: Prof. Dr. Name
Degree Program: Computer Science
Submission Date: April 8, 2026

Abstract

In this thesis, the focus is on exploring the performance of large language models (LLMs) on algorithmic programming tasks with the help of selected problems from the Advent of Code as a benchmark. Specifically, this thesis seeks to assess the generated codes by LLMs in both Python and Rust to measure not only the ability of LLMs to generate a functionally correct program but also the effect that the targeted programming language might have on output quality.

In order to make sure that this research is scientifically accurate, unbiased, and deterministic, all of the experiments in the thesis are going to be run using the same problem sets and the same approaches to prompting and model generation. In addition, the assessment will be performed by evaluating LLM outputs using different metrics such as correctness of a solution, its execution, the algorithm used and memory performance, as well as, compilation rate, and code quality in terms of structure and style of writing within the target programming language.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

More and more, Large language models are being used for a variety of software engineering tasks, such as code generation, bug fixes and refactoring. As the use of such systems becomes more generalized and their role in industrial practices continues to spread, it is important to evaluate the performance of these systems on both correctness of code, and efficiency in terms of runtime and memory usage. Predefined algorithmic programming tasks offer such a basis and are a suitable setting for such an evaluation. In this setting, these models have to interpret a natural language problem, come up with a suitable solution to that problem, and translate that solution into working code. This task requires structured reasoning, precise input parsing and a consideration of algorithmic complexity.

1.2 Problem Statement

Although large language models can often generate plausible code, their actual performance on algorithmically demanding tasks remains uncertain. It is therefore important to investigate:

- how often they generate correct solutions,
- whether they select efficient algorithms,
- how robust their outputs are across different programming languages, and
- what kinds of errors occur in practice.

This thesis addresses these questions using Advent of Code as the evaluation benchmark and comparing generated solutions in Python and Rust.

1.3 Research Objectives

The main objective of this thesis is to benchmark selected large language models on algorithmic programming tasks and evaluate their generated solutions. The thesis focuses on:

- correctness,
- runtime performance,
- memory usage,
- compilation success,
- and the structure of the generated code.

1.4 Research Questions

The main research question is:

> How effectively can large language models solve algorithmic programming tasks from Advent of Code, and how do their generated solutions differ between Python and Rust?

Sub-questions include:

- What proportion of selected tasks can be solved correctly?
- How do runtime and memory usage differ between Python and Rust solutions?
- What kind of algorithms are preferred and are those the most efficient option?
- Does the target programming language influence correctness, compile success, or code quality?

Chapter 2

Background

2.1 Large Language Models

Large Language Models are neural networks that are trained on large corpora of text data, by employing a Transformer architecture, first proposed by Google in 2017 in the paper "Attention is all you need". Compared to the previous approaches to sequence modelling, such as recurrent and convolutional neural networks, Transformer architecture relies heavily on self-attention mechanisms to capture long range dependencies and scale training to large data sets. Recent models have shown impressive results in a variety of software engineering applications, ranging from code generation and explanation, to error and bug fixes as well as refactoring. The effectiveness of LLMs in solving programming tasks is highly dependant on pretraining on datasets containing texts and programming code collected from different sources. Such training allows these models to learn language structure and programming syntax, as well as API libraries, different writing styles and common coding patterns. However, the generated code must not be confused with effective problem solving, Although LLM's can easily generate code that seems syntactically correct and reasonable, they may fail to solve certain problems that involve more complicated reasoning, constraints or algorithms.

2.2 Algorithmic Programming Tasks

In contrast to coding challenges, where a model needs to fill gaps in the existing code, algorithmic problems require understanding of a whole problem statement and turning it into a right and optimized computation. There is a number of specific steps involved, including:

- parsing input data,
- understanding the underlying computational problem,

- designing a suitable algorithm,
- implementing the solution correctly,
- and reasoning about complexity.

Typical categories include graph problems, dynamic programming, simulation, greedy methods, search, and string processing.

2.3 Advent of Code

The advent of code (AoC) is an annual coding competition that releases 25 coding challenges over 25 days in December (as of 2025, there are 12 coding challenges) **aoc**. Every challenge has two parts; part two is generally an extension of part one in terms of constraints or solution techniques. AoC problems have a variety of themes, ranging from basic arithmetic to complex algorithms. The difficulty of the problem statements also varies, making AoC a good measure of a code generation system's ability to solve problems.

AoC challenges tend to involve writing entire programs instead of small code segments. It requires comprehending the natural language description of a problem and developing an appropriate approach, which needs to be implemented using programming skills. Given the need for reasoning and implementing the solution within a programming language, AoC problems make excellent candidates for evaluating algorithmic problem solving in large language models.

2.4 Programming Languages: Python and Rust

This thesis compares generated code in Rust and Python, to figure out how the different languages affect the generated solutions. In the two languages there are significant differences in terms of abstraction, execution and type safety, as well as runtime and memory usage.

2.4.1 Python

Python is a popular programming language known for its simplicity and ease of use. As an interpreted language, Python is suitable for scripting and application development. Its high level features include clear syntax, the ability to utilize high level data structures and different object oriented features. Python has a number of built in types, including lists, dictionaries and tuple sets, allowing a broad set of patterns to be implemented. Python as an interpreted language is dynamically typed, meaning the errors in the code are not revealed until the code is executed. Since Python is not compiled, running a program takes significantly longer to execute, and requires a lot more memory. As a result, Python

code may be easier for an LLM to generate correctly, however this code may be a lot worse in a runtime perspective.

2.4.2 Rust

Rust is a systems programming language, designed for high performance and safety. A distinctive feature of the programming language is its ownership concept coupled with borrowing and lifetime. With those aspects, rust can implement certain aspects of memory safety at compile time. In addition, rust implements a zero-cost abstraction approach, meaning that higher level language concepts should compile to code that is similar in performance to low level implementations. For this thesis Rust is interesting because its structural constraints are far stronger than those in python. For a working code, the LLM not only needs to figure out the algorithmic problem but also needs to meet certain criteria set by the compiler, such as ownership, mutability, borrowing and type correctness. Since Rust is a compiled language that is specifically designed to be efficient in runtime and memory usage, its comparison to python should be already clear beforehand, however, considering Rust is a rather new programming language, this might impose a bit more of a challenge to generate working, and the most efficient code.

2.5 Related Work: LLMs for Code Generation

Earlier work has shown that large language models are able to produce executable programs for various tasks, for example, early models like Codex performed extremely well in the benchmarks for code generation, particularly HumanEval where the criteria mostly centres around functional correctness by passing multiple unit tests given in natural language descriptions. Another line of work evaluated the abilities of such models in benchmarks based on textual task descriptions, such as MBPP and MathQA-Python. Subsequent work further explored code generation for more challenging tasks, especially AlphaCode explored the generation of competition level code and showed that harder algorithmic tasks remained hard to solve by LLMs, since they require significant reasoning and more complicated solutions

Chapter 3

Methodology

3.1 Study Design

In this thesis, a benchmark is developed to measure the performance of the code generation systems against each other when performing algorithmic programming. For this purpose, two models are tested for two programming languages with the use of the same set of problems from *Advent of Code*. By limiting variation to specific experimental factors and keeping others consistent across tests, the attempt is made to create as precise and coherent comparison as possible.

In this benchmark, the independent variables include:

- the model,
- the programming language, and
- the level of problem difficulty.

The dependent variables are those observed results that will be used to evaluate the performance:

- correctness,
- compileability,
- execution time,
- algorithm,
- memory usage, and
- the number of lines of code.

Thus, in addition to the successful completion of a task by a model, the performance measurement can include information on the efficiency and structural complexity of the generated program code.

3.2 Scientific Benchmarking Principles

For ensuring the scientific validity of the benchmarking experiment, certain fundamental properties of reproducibility, fairness, objectivity, and transparency are followed.

3.2.1 Reproducibility

The experiments performed are done in well-controlled and consistent environments. In each experiment, identical machines are used along with identical operating systems, template prompts, and evaluation scripts. This ensures that no other factors other than those being studied impact the results of the experiment. For the purpose of reproducibility, the repository associated with the benchmark includes the chosen problems, template prompts, output code, and result tables.

3.2.2 Fairness

Each model used for evaluation is presented the same prompt format under the same conditions. In all the experiments performed, only the problem file, input file, and target programming language are varied according to the requirement of the benchmark. Moreover, no extra hints or information specific to a model is provided for the generation process. Every model has two attempts at generating the correct solution, once with the standard prompt and then a second time with an error prompt, where it states the error.

3.2.3 Objectivity

All code generations will be objectively assessed by pre-defined scripts and criteria. The generated code is run as-is without any human interference or manual correction. Any bias brought in by the subjective correction or modification is therefore eliminated. Should repairing the generated code be necessary, it would only be done via a predefined repair prompt and tracked independently of the original generation, thereby making it clear whether the results were achieved through initial generation or the repair prompt.

3.2.4 Transparency

The framework allows all experimental runs to be transparently documented in such way that they may later be examined. At least the following elements are tracked in every experiment:

- precise prompt used,
- model used,
- generated code,

- number of attempts
- assessment metrics.

3.3 Problem Selection

A specific set of *Advent of Code* problems will be chosen prior to the experiments. Defining the problems ahead of time will help to avoid any possible selection bias and make sure that all of the assessed models are going to be evaluated based on exactly the same criteria. For constructing an unbiased benchmark, there should be a number of problems of different difficulties and belonging to different classes.

Namely, the benchmark should consist of problems requiring different skills from the model, such as:

- parsing and string manipulation,
- simulations,
- graphs,
- state tracking, and
- optimization and dynamic programming.

The importance of choosing various types of problems lies in the fact that not all of the problems are equally demanding towards one specific skill. Some problems may demand accurate processing of input and adherence to the rules of operation, while some other problems rely more heavily on proper choice of algorithm or data structure.

3.4 Models, Languages and Enviroment

The benchmark compares two code-generation large language models:

- OpenAI Codex (v0.117.0) with the model gpt-5.4
- Antrophic Claude Code (v2.1.91) with the model Sonnet 4.6

Each model is evaluated in:

- Python (v3.14.2)
- Rust (v1.94.1)

Each code is produced and tested within a Ubuntu WSL terminal in VSCode

In total there were 12 problems considered, which equates to a total of 48 runs, before any repair attempts were considered

3.5 Prompt Design

A standardized prompt template is used across all runs. Only the problem file, input file, and target language are changed. The benchmark prompt is intentionally short, explicit, and easy to automate.

```

Solve the programming problem described in:
{PROBLEM STATEMENT}

Requirements:
- Language: {LANGUAGE}
- Read input from: {INPUT}
- Put the code into the file: {OUTPUT}
- No external libraries
- Handle full input correctly

Output of the code should only be the required answer!

Rules:
- Output the complete source code
- Ensure the solution is correct and reasonably efficient
- Follow idiomatic style for the chosen language
- Make reasonable assumptions based only on the given input, do
   not invent missing requirements
```

3.6 Repair Prompt

If the initial generated code fails, one repair attempt is given, for this the following repair prompt is used:

```
The previous solution failed.

Error:
{ERROR_MESSAGE}

Fix the solution.

Requirements:
- Same input/output format
- Output ONLY corrected source code
```

3.7 Evaluation Metrics

The following metrics are evaluated during each benchmark test execution, to inspect the quality of the solution. In total, these metrics evaluate if the program solves the problem, whether it compiles, how efficient it is and how much memory it uses, and the design of the algorithm.

3.7.1 Correctness

Correctness is determined based on the comparison between the output of the generated code and the correct solution that corresponds to the given Advent of code problem. Only at an exact match is the code considered correct. [TODO: Part A & Part B determination]

3.7.2 Compile Success

As for Rust, compile success can be considered as a binary metric, indicating whether the program compiled successfully or not. Certain compile-time checks have to be met, in order to ensure successful compilation.

3.7.3 Runtime

Runtime is measured in order to figure out, how efficient the solution is. To minimize the impact of any randomness resulting from background activities, scheduling or any environmental factors, each programm is run several times. In the benchmarking test, the average of all the runs is given, as well as minimum, maximum and the standard deviation

3.7.4 Memory Usage

Memory usage is measured with the GNU time utility, where for this benchmark the average memory usage, the maximum and the minimum is being recorded across a given number of runs. GNU time provides a standardized system level measurement that can be applied to all runs.

3.7.5 Lines of code

The number of lines of code is used as a structural measurement that reflects the implementation size and verbosity of the program. This way it can be determined across the different models, if one might generate a more elegant version, or a more verbose or redundant solution.

3.7.6 Algorithmic Evaluation

In addition to testing the correctness and performance of the generated solutions, the algorithms or the code are also evaluated for their quality. This evaluation checks what algorithm the model used, and if this is the optimal choice for the given problem

3.8 Execution Procedure

Each experiment follows the same pipeline:

1. Submit the prompt to the selected model.
2. Save the raw generated code.
3. Compile the solution if required.
4. Execute the program on the fixed input.
5. Record output, runtime, memory usage, and errors.
6. Compare the output to the expected results.

3.9 Failure Rules

The following outcomes are defined before running the benchmark:

Table 3.1: Failure rules

Case	Interpretation
Compile fails	failure
Runtime error	failure
Timeout	failure
Wrong output	incorrect
Output format incorrect	failure

3.10 Threats to Validity

There are several potential threats to the validity of the results. For example, LLMs are generally vulnerable to even small changes in prompt wording and formatting, which implies that even small changes in the input can alter the output. Also, the models performance can vary over time due to updates and replacements of the existing models. Lastly, although Advent of Code provides a useful source of algorithmic programming challenges,

it only represents one style of programming task and therefore does not represent more types of possible software engineering tasks.

Chapter 4

Implementation

4.1 Benchmark Environment

All experiments are executed on the same hardware and software environment. The thesis should document:

- CPU,
- RAM,
- operating system,
- Python version,
- Rust version,
- and any benchmark tools used.

4.2 Repository Structure

A reproducible repository structure helps keep the benchmark organized.

```
\begin{lstlisting}
aoc-llm-benchmark/
|-- prompts/
|-- generated/
|   |-- codex/
|   |   |-- python/
|   |   |   |-- 2024/
|   |   |   |-- 2025/
|   |   |-- rust/
|   |   |   |-- 2024/
|   |   |   |-- 2025/
```

```
|   |-- claude/
|   |   |-- python/
|   |   |   |-- 2024/
|   |   |   |-- 2025/
|   |   |-- rust/
|   |   |   |-- 2024/
|   |   |   |-- 2025/
|-- results/
|-- thesis/
```

Each year directory contains the `problems`, `solutions`, and `benchmarking-test`.

4.3 CLI-Based Generation Workflow

The benchmark is performed using the command-line interfaces of both the evaluated systems to ensure that all runs can be performed in a consistent way, which enables scripting all runs. The use of command-line access allows for automating the entire process of benchmarking and minimizing the possibility of human error during the process of data collection.

The general workflow includes the following steps:

1. presenting a common prompt to both systems,
2. writing the produced code into the respective output files,
3. compiling and executing the produced solution automatically, and

4.4 Validation and Measurement

Correctness is validated against expected outputs stored for each Advent of Code problem. Runtime is measured using repeated executions and summarized using the median value. Memory usage is collected using the GNU time function. Lines of code are counted directly from the generated source files.

4.5 Example Data Collection Table

A structured Excel sheet is used to document all the results like in the following example:

Table 4.1: Example benchmark data schema

Problem	Model	Language	Correct	Runtime	Memory	LOC
Day01	Codex	Python	1	0.12	15 MB	45
Day01	Codex	Rust	1	0.03	6 MB	62
Day01	Sonnet	Python	0	0.18	17 MB	48
Day01	Sonnet	Rust	0	–	–	58

Chapter 5

Results

5.1 Overview

This chapter presents the benchmark results for correctness, runtime, memory usage, compile success, code size and the algorithmic evaluation.

5.2 Correctness

[TODO: Add results]

5.3 Compile Success

Compile success is only relevant for Rust. [TBD if this is necessary]

[TODO: add results]

5.4 Runtime

Runtime measurments are reported with the average value, maximum, minimum and standard derivation.

[TODO: add results]

5.5 Memory Usage

Memory usage is also being measured by average, maximum and minimum

[TODO: add results]

5.6 Code Size

Lines of code in order to evaluate the structure and compare it between models
[TODO: add results]

5.7 Algorithmic Evaluation

This section determines what algorithm has been used and if this is the best choice for the given problem

5.8 Figures

[TODO: plots for visual comparison]

5.9 Possible section: Generation time

The time it took to generate each code [TBD!]

Chapter 6

Discussion

6.1 Interpretation of Results

[TODO]

- Which model solved more tasks correctly?
- Did one model perform better on specific problem categories?
- Were Python solutions more reliable than Rust solutions?
- Did runtime advantages of Rust appear consistently?

...

6.2 Model Comparison

The comparison between Codex and Sonnet in both quantitative and qualitative outcomes [TODO]

6.3 Language Comparison

[TODO]

6.4 Limitations

- only a limited number of Advent of Code tasks are included,
- models may change over time,
- prompt phrasing may still influence results,

- and the benchmark focuses on one class of programming problems.

[TODO]

Chapter 7

Conclusion

7.1 Summary

[TODO]

7.2 Answer to the Research Question

[TODO]

Appendix A

Appendix